

FUNDACIÓN FULGOR
Protocolos de Comunicación

Informe del Trabajo Práctico 3

Laboratorios de capa física Ethernet: MAC, XGMII, PCS 64b/66b y MLD con simulación
hardware-in-the-loop

Alumno: Francisco Javier Vasquez Curetti

Docentes: Facundo Sposetti, Jorge Manuel Finochietto

Septiembre de 2026

Índice

1	Introducción	1
2	Entorno de trabajo	1
3	Lab 1: trama Ethernet cruda (bus de 8 bits)	2
3.1	Objetivo	2
3.2	Diseño	2
3.3	Resultados	2
4	Lab 2: interfaz XGMII de 64 bits	3
4.1	Objetivo	3
4.2	Resultados	4
5	Lab 3: PCS 64b/66b y scrambler	5
5.1	Objetivo	5
5.2	Diseño	5
5.3	Resultados: codificación 64b/66b	6
5.4	Resultados: scrambler	7
6	Lab 4: PCS 100GBASE-R y Multi-Lane Distribution (MLD)	8
6.1	Objetivo	8
6.2	Mecanismo real (cláusula 82) y abstracción del laboratorio	8
6.3	Ejecución	9
6.4	Respuestas a la consigna	9
7	Conclusiones generales	10

1 Introducción

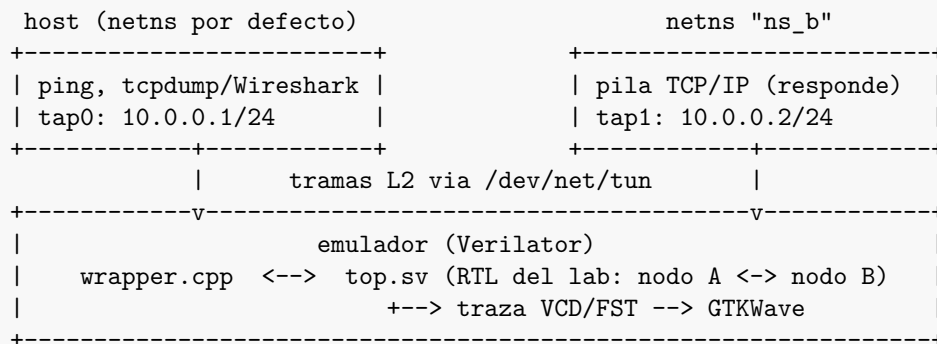
El Trabajo Práctico 3 recorre, en cuatro laboratorios, las subcapas de la capa física de Ethernet de alta velocidad definidas en IEEE 802.3: desde la trama de capa 2 “cruda” hasta la distribución en múltiples carriles de 100GBASE-R. A diferencia de los TP anteriores, el RTL (SystemVerilog) y el entorno de simulación (C++ con Verilator) fueron provistos por la cátedra en el repositorio `ethernet-phy-labs`. El trabajo consistió en ejecutar cada laboratorio con tráfico real, capturarlo y correlacionar lo que se ve a nivel de paquetes (Wireshark) con lo que ocurre ciclo a ciclo dentro del hardware (GTKWave).

Tabla 1: Resumen de los cuatro laboratorios.

Lab	Subcapa (IEEE 802.3)	Unidad en el bus	Qué hace el RTL	Patrón de ping
1	Trama L2 cruda	1 byte por ciclo	Un registro por sentido	cafe0001
2	RS / XGMII (cl. 46)	64 bits + 8 de control	Registra palabras XGMII y extrae campos	cafe0001
3	PCS 64b/66b (cl. 49/82)	Bloques de 66 bits	MAC + codificador/decodificador + scrambler	cafe0003
4	PCS + MLD (cl. 82)	66 bits + banda lateral	MAC + PCS + distribución en 20 lanes virtuales	feca0004

2 Entorno de trabajo

Los cuatro laboratorios comparten la misma metodología *hardware-in-the-loop* (HIL): el kernel de Linux genera tráfico real que atraviesa el modelo RTL simulado.



- **Red virtual.** `setup_netns.sh` crea el namespace `ns_b` y dos interfaces TAP: `tap0` en el host (10.0.0.1/24, MAC 02:ca:ee:44:4a:fa) y `tap1` dentro de `ns_b` (10.0.0.2/24, MAC b6:cb:dc:76:cc:85). Como están en namespaces distintos, el tráfico entre ambas direcciones no puede resolverse dentro del kernel: necesariamente pasa por el emulador.
- **Emulador.** Verilator compila `top.sv` junto con `wrapper.cpp`. El wrapper lee las tramas de cada TAP, las inyecta en los puertos del RTL, reconstruye las tramas de salida, las escribe en la TAP opuesta y vuelca la traza de señales.
- **Ejecución.** En tres terminales: el emulador, `tcpdump` sobre `tap0` y `ping` con un patrón de datos propio de cada lab (`-p`), fácil de reconocer en hexadecimal. Después, Wireshark para la captura y GTKWave para la traza.

```

sudo ./scripts/setup_netns.sh                                # namespace + interfaces TAP
# compilación: --trace-fst (FST) en los labs 3-4; --trace (VCD) en los labs 1-2
verilator --cc --trace-fst --exe --build wrapper.cpp top.sv -o emulator
sudo ./obj_dir/emulator                                      # terminal 1
sudo tcpdump -i tap0 -w captura.pcapng                      # terminal 2
ping -c 2 -p cafe0001 -I tap0 10.0.0.2                     # terminal 3

```

Dos convenciones sirven para leer todas las trazas:

- **Escala de tiempo.** El wrapper avanza el tiempo de simulación una unidad por cada medio período y la traza usa `timescale 1ps`, por lo que en GTKWave **un ciclo de reloj equivale a 2 ps**. Son tiempos nominales, no físicos.
- **Orden de bytes.** En los buses de 64 bits, el lane 0 (primer byte transmitido) ocupa los bits [7:0]. GTKWave muestra el valor en hexadecimal con el bit más significativo a la izquierda, así que cada palabra se lee “al revés”: de derecha a izquierda, de a un byte.

3 Lab 1: trama Ethernet cruda (bus de 8 bits)

3.1 Objetivo

Montar el pipeline HIL básico y reconocer la anatomía de una trama Ethernet II tal como el kernel la entrega a una interfaz TAP: dirección destino (6 B), dirección origen (6 B), EtherType (2 B) y payload. A este nivel **no hay preámbulo, SFD ni FCS**: en hardware real los agrega y los quita el MAC/PHY. El objetivo es encontrar esos mismos bytes, uno por ciclo, dentro de la traza.

3.2 Diseño

`top.sv` es mínimo: dos registros independientes ($A \rightarrow B$ y $B \rightarrow A$) de 8 bits de dato más un bit de *valid*, sin ninguna lógica de *framing*. Toda la interpretación de la trama la hace Wireshark; el hardware solo transporta bytes. Los puertos reales son `a_data_in/a_valid_in`, `b_data_out/b_valid_out` (y sus simétricos), no los `eth_txd/eth_tx_en` que menciona la consigna.

3.3 Resultados

Con `ping -c 2 -p cafe0001` se obtuvieron 2 de 2 respuestas (0 % de pérdida) y `tcpdump` registró 12 paquetes en `tap0` (figura 1). Además de las dos parejas *Echo Request/Echo Reply* aparecen ARP en ambos sentidos y tráfico propio del sistema (ICMPv6 *Router Solicitation* y mDNS), porque la versión del script de red usada en este lab todavía no deshabilitaba IPv6.

```

francisco-javier-vasquez@francisco-javier-vasquez-Latitude-3410:~/Documentos/protocolos...
$ cd Documentos/protocolos/TP3/repo_if/ethernet-phy-labs/lab1-raw-ethernet
$ lab1-raw-ethernet ping -c 2 -p cafe0001 -i tap0 10.0.0.2
PATTERN: 0xcafe0001
PING 10.0.0.2 (10.0.0.2) from 10.0.0.1 tap0: 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=15.5 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=10.7 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 10.686/13.073/15.461/2.387 ms
$ lab1-raw-ethernet

francisco-javier-vasquez@francisco-javier-vasquez-Latitude-3410:~/Documentos/protocolos...
$ cd Documentos/protocolos/TP3/repo_if/ethernet-phy-labs/lab1-raw-ethernet
$ lab1-raw-ethernet tcpdump -i tap0 -w lab1_capture.pcap
tcpdump: tap0: You don't have permission to perform this capture on that device
socket: Operation not permitted
$ lab1-raw-ethernet sudo tcpdump -i tap0 -w lab1_capture.pcap
[sudo] contraseña para francisco-javier-vasquez:
tcpdump: listening on tap0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
C12 packets captured
2 packets received by filter
0 packets dropped by kernel
$ lab1-raw-ethernet

```

Figura 1: Lab 1: ping con patrón cafe0001 (izquierda) y captura con `tcpdump` sobre `tap0` (derecha).

En Wireshark (figura 2), el *Echo Request* ocupa 98 bytes: destino `b6:cb:dc:76:cc:85` (`tap1`), origen `02:ca:ee:44:4a:fa` (`tap0`), EtherType `0x0800` (IPv4) y datos ICMP con `ca fe 00 01` repetido. Las tramas ARP miden 42 bytes, por debajo del mínimo Ethernet de 60, precisamente porque a nivel TAP no hay relleno ni FCS. En el *Echo Reply* las direcciones aparecen intercambiadas.

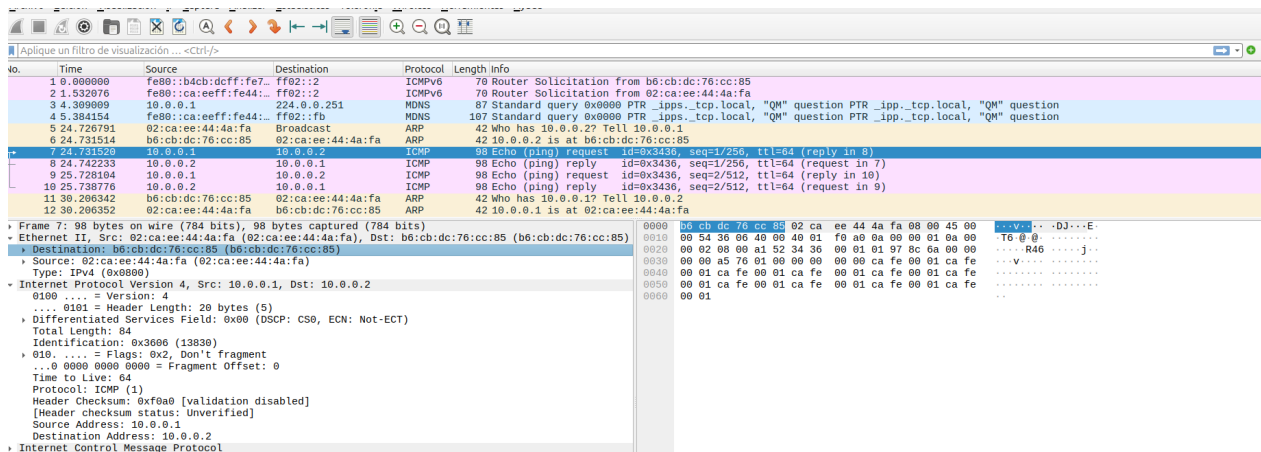


Figura 2: Lab 1: *Echo Request* en Wireshark: direcciones MAC, EtherType 0x0800 y el patrón ca fe 00 01 en el volcado hexadecimal.

En GTKWave (figura 3), mientras `a_valid_in = 1`, el bus `a_data_in[7:0]` recorre B6 CB DC 76 CC 85 | 02 CA EE 44 4A FA | 08 00 | 45 00 ..., exactamente los bytes del volcado de Wireshark y en el mismo orden, uno por ciclo. Más adelante aparece el patrón CA FE 00 01. Una trama de 98 bytes ocupa 98 ciclos del bus. Cuando termina la *Request* (`a_valid_in` vuelve a 0), el *Reply* entra por `b_data_in` con destino y origen invertidos (02 CA EE 44 4A FA B6 CB DC 76 CC 85 08 00 ...).

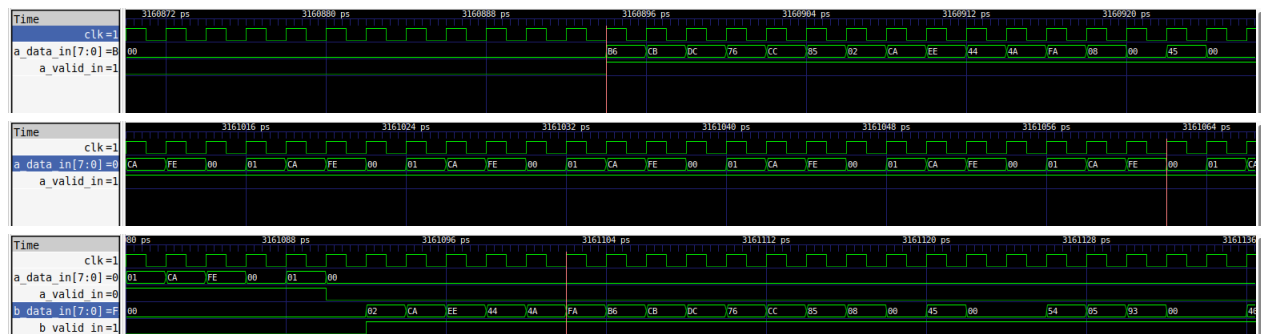


Figura 3: Lab 1: `a_data_in` en GTKWave. Arriba: inicio de trama (MAC destino, MAC origen, EtherType, inicio de IPv4). Centro: datos ICMP CA FE 00 01. Abajo: fin del *Request* y comienzo del *Reply* en `b_data_in`.

Latencia. El RTL agrega un único registro por sentido. Sin embargo, en la traza `b_data_out` toma cada byte en el mismo flanco en que aparece en `a_data_in` (se verificó sobre el VCD). La razón es que el wrapper actualiza la entrada y aplica el flanco de subida en la misma llamada a `eval()`, así que el registro captura el dato nuevo en ese mismo flanco y la traza no muestra desfase entre entrada y salida.

Conclusión. La trama que muestra Wireshark es exactamente la secuencia de bytes que atraviesa el RTL. Este laboratorio fija el método del resto del TP: ubicar en la traza, byte a byte, lo que se ve en la captura.

4 Lab 2: interfaz XGMII de 64 bits

4.1 Objetivo

Pasar de un byte por ciclo a la interfaz XGMII de 10G (IEEE 802.3, cláusula 46): 8 lanes de 8 bits (`TXD[63:0]`) más un bit de control por lane (`TXC[7:0]`, 1 = carácter de control). La trama se

delimita con caracteres de control: $/I/ = 0x07$ (idle), $/S/ = 0xFB$ (start, siempre en el lane 0) y $/T/ = 0xFD$ (terminate, en cualquier lane). Después de $/S/$ van el preámbulo ($0x55$) y el SFD ($0xD5$), y la trama termina con la FCS (CRC-32).

Versión utilizada. El lab se hizo con la versión del repositorio disponible en ese momento (lab2-xgmii-64bit). Al día siguiente la cátedra la reemplazó por lab2-cgmii, que implementa el MAC en RTL. En la versión usada, el *framing* XGMII lo arma `wrapper.cpp`: construye las palabras, calcula y agrega la FCS y la verifica en recepción. `top.sv` registra las palabras XGMII (una etapa por sentido) y extrae campos del encabezado para depuración. El análisis del bus XGMII es el mismo en ambas versiones.

4.2 Resultados

`ping -c 2 -p cafe0001` volvió a dar 2 de 2 respuestas y `tcpdump` registró 8 paquetes (mDNS, ARP, dos pares *Echo* e ICMPv6). La figura 4 muestra que el *Echo Request* es idéntico al del lab 1: la interfaz XGMII no modifica la trama.

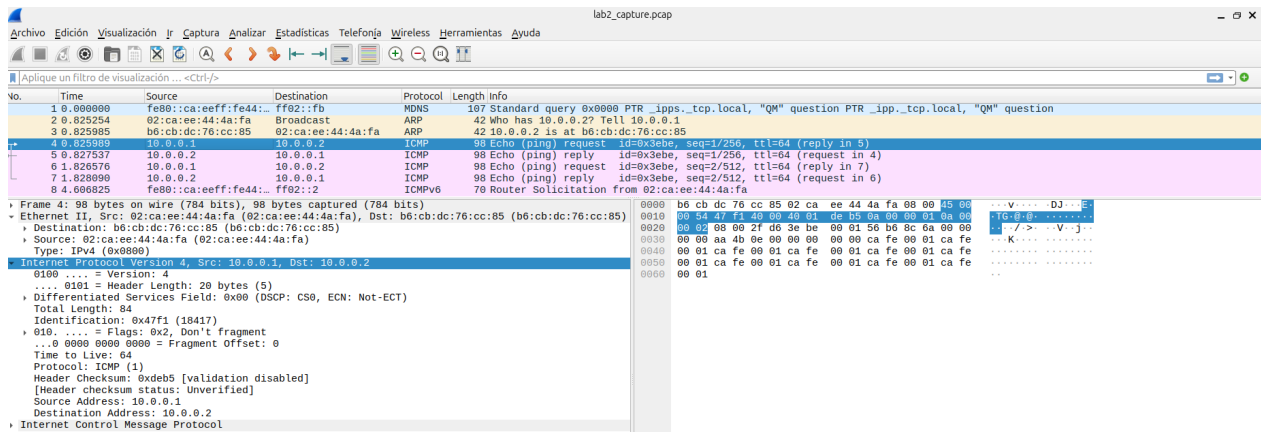


Figura 4: Lab 2: captura en Wireshark; *Echo Request* de 98 bytes con EtherType $0x0800$ y datos `ca fe 00 01`.

La tabla 2 sigue el *Echo Request* palabra por palabra sobre el bus de transmisión (figura 5, dos primeras franjas). La palabra 0 aparece en $t = 351532$ ps y la 13 en $t = 351558$ ps, una por ciclo.

Tabla 2: Lab 2: palabras XGMII del *Echo Request* en el bus de transmisión.

Palabra	a_txc	a_txd	Contenido, en orden de transmisión (lane 0 $\rightarrow$ 7)
idle	FF	0707070707070707	$8 \times /I/$ (idle)
0	01	D5555555555555FB	$/S/$ en el lane 0, $6 \times$ preámbulo 55 y SFD D5
1	00	CA0285CC76DCCBB6	b6 cb dc 76 cc 85 (destino) y 02 ca (inicio del origen)
2	00	00450008FA4A44EE	ee 44 4a fa (fin del origen), 08 00 (EtherType), 45 00 (IPv4)
3–12	00	(varía)	resto de IPv4/ICMP y datos; p. ej. FECA0100FECA0100 = 00 01 ca fe 00 01 ca fe
13	C0	07FDCF4531E30100	00 01 (últimos datos), FCS e3 31 45 cf, $/T/$ (lane 6), $/I/$ (lane 7)
idle	FF	0707070707070707	vuelta a idle

Observaciones:

- **Start.** TXC = $0x01$ marca como control solo el lane 0 ($/S/$); preámbulo y SFD viajan como datos.
- **Terminate.** La trama de 98 B más 4 B de FCS suma 102 B: 12 palabras completas y 6 bytes en la palabra final. Por eso $/T/$ cae en el lane 6 y TXC = $0xC0$ (lanes 6 y 7 de control). La trama completa ocupa 14 ciclos, frente a 98 en el lab 1: el bus es 8 veces más ancho.

- **FCS.** Se verificó fuera de línea que `e3 31 45 cf` coincide con el CRC-32 de IEEE 802.3 calculado sobre los 98 bytes de la trama capturada (valor `0xCF4531E3`, transmitido desde el byte menos significativo). Lo mismo con `7a 2b b1 ca` para el *Echo Reply*.
- **Respuesta.** En el bus de recepción (`a_rxd/a_rxc`, dos franjas inferiores de la figura 5) el *Reply* tiene la misma estructura, con destino y origen intercambiados (`CBB6FA4A44EECA02`, ...).
- **Latencia.** Igual que en el lab 1: un registro por sentido, sin desfase visible entre `a_txd` y `b_rxd` en la traza.

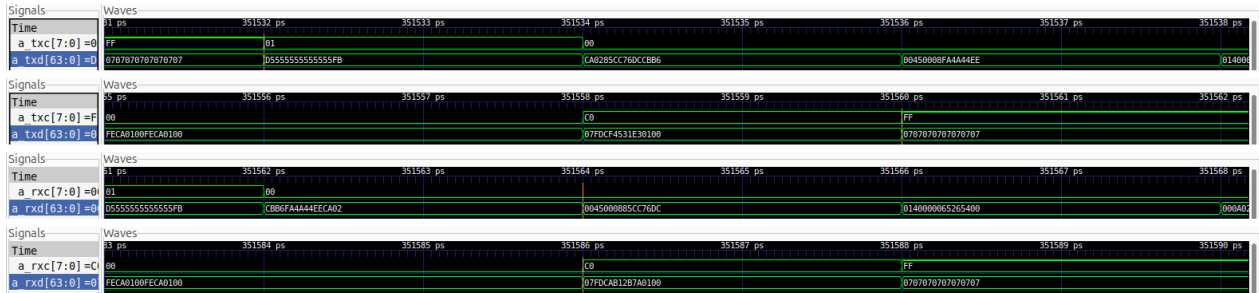


Figura 5: Lab 2: bus XGMII en GTKWave. De arriba hacia abajo: inicio del *Request* (idle → start → encabezado), fin del *Request* (datos → FCS + /T/ → idle), inicio y fin del *Reply* en `a_rxd/a_rxc`.

Conclusión. XGMII agrega la noción de palabra de 64 bits con señalización de control fuera de banda (TXC). La delimitación de la trama ya no depende de un *valid*, sino de /S/ y /T/, y la FCS le permite al receptor validar la integridad.

5 Lab 3: PCS 64b/66b y scrambler

5.1 Objetivo

Estudiar la PCS de 10GBASE-R/100GBASE-R (IEEE 802.3, cláusulas 49 y 82). Cada palabra XGMII (64 bits de datos y 8 de control) se codifica en un **bloque de 66 bits**: un *sync header* de 2 bits y 64 bits de payload.

- **01: bloque de datos**, con 8 octetos de datos.
- **10: bloque de control o mixto.** Su primer octeto es el *Block Type Field* (BTF), que indica el formato del resto. Los caracteres de control se comprimen: cada /I/ pasa a un código de 7 bits `0x00`, y /S/ y /T/ quedan implícitos en el BTF.

El *overhead* es de 2 bits por cada 64 de datos, un 3,125 % (frente al 25 % de 8b/10b). Después, el payload pasa por un **scrambler autosincronizante** $G(x) = 1 + x^{39} + x^{58}$. El *sync header* no se aleatoriza.

5.2 Diseño

La topología es `host A <-> MAC A <-> CGMII <-> PCS A <=> (hdr + payload) <=> PCS B <-> MAC B <-> host B`. Ahora el MAC está en el RTL: arma la trama con preámbulo y SFD, la rellena hasta 60 B, calcula y agrega la FCS y genera /T/ e idles.

Dos diferencias con la documentación condicionaron el análisis:

- Todo está en un único `top.sv`, con nombres de señales distintos de los documentados. En el codificador (`top.pcs_a.encoder`), `raw_hdr` y `raw_payload` son el bloque combinacional antes del registro y del scrambler, y `hdr` y `payload` son la salida registrada (aleatorizada si el scrambler está activo). En el decodificador (`pcs_rx`) están `descram_payload` y `block_type`.

- El `wrapper.cpp` de esa versión no volcaba la traza, aunque los comandos de compilación de la consigna lo suponían. Se le agregó localmente el volcado FST. La cátedra lo corrigió después en el repositorio.

Se hicieron dos corridas: fase 1 sin scrambler (valor por defecto) y fase 2 con `--enable-scrambler`.

5.3 Resultados: codificación 64b/66b

Con `ping -c 2 -p cafe0003` se obtuvieron 2 de 2 respuestas. La captura (figura 6) tiene 8 paquetes: un intercambio ARP, dos pares *Echo* y otro intercambio ARP iniciado por `ns_b`. Ya no hay tráfico IPv6, porque el script de red actualizado lo deshabilita. Un detalle: el ARP generado por `ns_b` llega a `tap0` con **60 bytes**, mientras que el que envía el host mide 42. El MAC TX del RTL rellena hasta el mínimo de 60 B y el MAC RX solo quita la FCS, no el relleno.

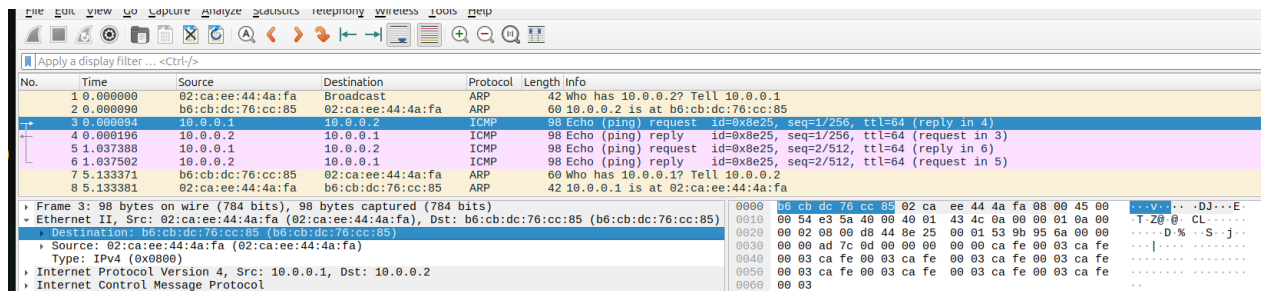


Figura 6: Lab 3: captura en Wireshark (fase 1). El ARP que llega desde `ns_b` mide 60 bytes por el relleno del MAC del RTL.

Tabla 3: Lab 3: bloques generados por el codificador de la PCS A (fase 1, sin scrambler).

Bloque	cgmii_txc	cgmii_txd	raw_hdr	raw_payload
Idle	FF	0707070707070707	10	0000000000000001E
Start	01	D555555555555555FB	10	D5555555555555578
Datos	00	CA0285CC76DCCBB6	01	CA0285CC76DCCBB6
Terminate	C0	07FD15B839350300	10	0015B839350300E1

La tabla 3 y la figura 7 muestran los cuatro tipos de bloque que genera el codificador:

- **Idle.** BTF 0x1E (ocho caracteres de control). Cada `/I/` se codifica como 0x00, por eso el resto del payload es cero.
- **Start.** BTF 0x78. `/S/` queda implícito y los 7 bytes restantes (preámbulo y SFD) viajan como datos.
- **Datos.** Sync 01 y payload igual a la palabra XGMII, sin BTF.
- **Terminate.** `/T/` en el lane 6 $\rightarrow$ BTF 0xE1. Se conservan los 6 octetos de datos (00 03 y la FCS 35 39 b8 15); el `/I/` del lane 7 pasa a 0x00.

Las salidas registradas `hdr` y `payload` repiten estos valores un ciclo después.

El BTF de cierre depende del lane en que cae `/T/`: 0x87, 0x99, 0xAA, 0xB4, 0xCC, 0xD2, 0xE1 y 0xFF para los lanes 0 a 7. Al recorrer toda la traza de la fase 1, cada codificador generó 4 bloques de *start* y 4 de *terminate*:

- **2 con 0xE1:** las tramas ICMP (98 + 4 = 102 B) dejan 6 bytes en la última palabra.
- **2 con 0x87:** las tramas ARP, rellenas a 60 B, más 4 de FCS suman 64 B, exactamente 8 palabras. `/T/` cae entonces en el lane 0 de una palabra nueva.

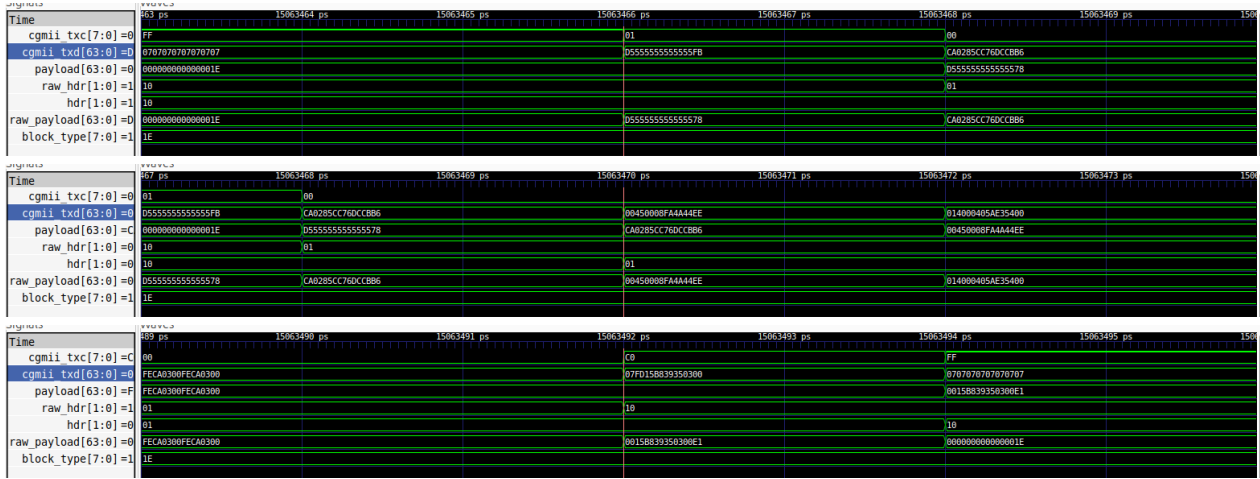


Figura 7: Lab 3, fase 1: codificación en GTKWave. Arriba: idle → start (BTF 0x78). Centro: start → bloques de datos (`raw_hdr` = 01). Abajo: datos → terminate (BTF 0xE1) → idle.

5.4 Resultados: scrambler

La captura de Wireshark es idéntica con y sin scrambler: la aleatorización se deshace por completo en la PCS B, antes de llegar al MAC y a la TAP. Solo la traza la pone en evidencia (figura 8):

- Los bloques idle, que sin scrambler valen siempre ...001E, pasan a verse pseudoaleatorios (6204FEC71D2BD3BD, 7C6E7CCA1E0BCCDE, ...). Esa es la finalidad del scrambler: secuencias largas de idles o de datos repetitivos darían pocas transiciones para la recuperación de reloj y un espectro con líneas marcadas y desbalance de continua.
- `hdr` no cambia (10/01). El *sync header* nunca se aleatoriza porque el receptor lo necesita para encontrar los límites de bloque (*block lock*) antes de desaleatorizar.
- En la PCS B, `descram_payload` reproduce el `raw_payload` de la PCS A con un ciclo de retardo, el del registro de salida del codificador (tabla 4).

Tabla 4: Lab 3, fase 2: payload antes y después del scrambler, y su recuperación en el receptor.

t (ps)	raw_payload (PCS A)	payload un ciclo después	descram_payload (PCS B)
10294160	00006A95BBB90100	CAA83350E5586F99	00006A95BBB90100
10294162	0000000B82B30000	ACCE46A3953FC9CC	0000000B82B30000
10294164	FECA0300FECA0000	96AE4CCC17BDAEED	FECA0300FECA0000
10294166	FECA0300FECA0300	E5D9627899B293B0	FECA0300FECA0300

Las dos últimas filas muestran que **una misma palabra de entrada produce salidas distintas**, porque la salida depende de la historia del flujo. Además, se verificó fuera de línea, con la ecuación $s_i = d_i \oplus s_{i-39} \oplus s_{i-58}$, que cada `payload` es exactamente la aleatorización del `raw_payload` del ciclo anterior, y que aplicar el desaleatorizador devuelve el valor original.

Como el desaleatorizador arma su estado con los mismos bits aleatorizados que recibe, no hace falta compartir una semilla: se sincroniza solo después de 58 bits. Como contrapartida, un error de bit en la línea se multiplica por 3 a la salida (posiciones i , $i + 39$ e $i + 58$).

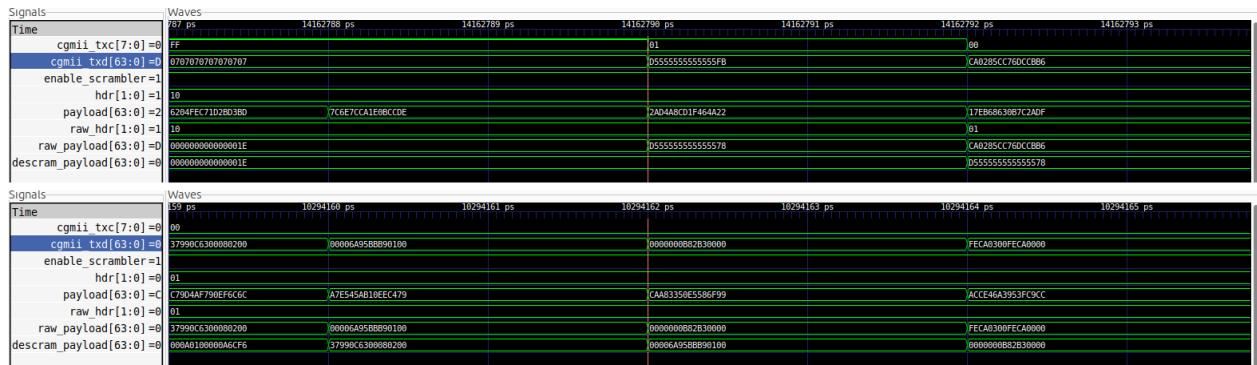


Figura 8: Lab 3, fase 2 (con scrambler): arriba, bloques idle aleatorizados y transición a start; abajo, bloques de datos (raw_payload, payload y descram_payload).

Conclusión. La PCS cambia por completo la representación de los datos en el enlace (bloques de 66 bits aleatorizados), pero es transparente para las capas superiores: Wireshark ve exactamente las mismas tramas.

6 Lab 4: PCS 100GBASE-R y Multi-Lane Distribution (MLD)

6.1 Objetivo

Entender cómo 100GBASE-R reparte el flujo de bloques de 66 bits entre **20 lanes virtuales (PCS lanes)** e inserta periódicamente **Alignment Markers (AM)** para que el receptor identifique cada lane, compense el *skew* y reordene. La consigna pide verificar la distribución *round-robin*, la periodicidad y el formato de los AM, y la propagación de las señales laterales del transmisor al receptor.

6.2 Mecanismo real (cláusula 82) y abstracción del laboratorio

En 100GBASE-R real, los bloques se distribuyen *round-robin* sobre 20 lanes. Cada 16383 bloques por lane se inserta un AM en **todos** los lanes a la vez; la PCS compensa ese espacio eliminando idles. Cada AM tiene 8 octetos, M0 M1 M2 BIP3 M4 M5 M6 BIP7, donde M4..M6 son el complemento de M0..M2 y BIP7 el de BIP3. Por ejemplo, el lane 0 usa C1 68 21 y 3E 97 DE. BIP3 es una paridad por intercalado de bits calculada sobre el lane desde el AM anterior. El receptor bloquea cada lane, lo identifica por su AM, alinea los lanes (*deskew*) y los reordena.

El laboratorio simplifica este mecanismo para que se pueda seguir en GTKWave (tabla 5).

Tabla 5: Lab 4: MLD real frente a la abstracción del laboratorio.

Aspecto	Cláusula 82	RTL del laboratorio
Distribución	20 lanes (demultiplexado real)	un único flujo de bloques, sin dividir
Intervalo de AM	16383 bloques por lane	AM_INTERVAL = 64 ciclos (parámetro)
Inserción del AM	reemplaza bloques (compensando con idles)	señal lateral: el bloque de datos nunca se reemplaza
Identificación del lane	implícita en el patrón de 8 octetos	explícita: mld_lane_id y un byte dentro de mld_am_pattern
BIP3/BIP7	paridad real, detecta errores	no implementado (relleno fijo 00 A5 A5 A5 A5 A5)
Deskew y reordenamiento	FIFOs por lane en el receptor	innecesario: no hay skew por construcción

El núcleo del transmisor MLD (mld_tx) es un contador de lane y un contador de bloques, ambos registrados en el mismo always_ff que el bloque de salida (versión resumida del código):

```

mld_lane_id    <= lane_ptr;                                // lane del bloque actual
lane_ptr       <= (lane_ptr == NUM_LANES-1) ? 0 : lane_ptr + 1;
mld_am_pattern <= {2'b10, 8'hC1, 8'(lane_ptr), 48'h00_A5_A5_A5_A5};
mld_is_am      <= (block_cnt == AM_INTERVAL-1);           // pulso cada AM_INTERVAL
block_cnt      <= (block_cnt == AM_INTERVAL-1) ? 0 : block_cnt + 1;
pcs_block_out  <= pcs_block_in;                            // (o aleatorizado)

```

6.3 Ejecución

El emulador se compiló con `-GAM_INTERVAL=64 -GNUM_LANES=20`; en esta versión la traza se genera siempre. `ping -c 5 -p fec80004` obtuvo 5 de 5 respuestas (0 % de pérdida) y `tcpdump` capturó 12 paquetes: ARP y cinco pares *Echo* (figura 9).

Con el orden de lanes de la sección 2, los datos ICMP `fe ca 00 04 ...` aparecen en el bus como bloques `1CAFE0400CAFE0400` (sync 01 seguido del payload). En esta versión:

- El codificador ubica el BTF en los bits [63:56]. Por ejemplo, un bloque idle se ve como `21E000000000000000` (sync 10 seguido de 1E), a diferencia del lab 3, donde el BTF ocupa [7:0].
- El MAC del RTL está simplificado: no agrega FCS ni relleno. Por eso el ARP de respuesta llega con 42 B, y la trama ICMP de 98 B cierra con BTF `0xAA` (/T/ en el lane 2) en lugar de `0xE1`.

```

+ lab4-pcs-mld sudo ./obj_dir/emulator
[TAP] Bound successfully to interface: tap0
[TAP] Bound successfully to interface: tap1
=====
[EMULATOR] Lab 4 Dual TAP Node A (tap0) <-> Node B (ns_b/tap1) Active
[EMULATOR] Waveform Dumping: ACTIVE (dump.fst)
[EMULATOR] Press Ctrl+C to stop simulation cleanly.
=====
^C
[EMULATOR] Flushing waveform buffers...
[EMULATOR] Saved waveform to dump.fst successfully.
+ lab4-pcs-mld

+ sudo tcpdump -i tap0 -w ~/Documentos/protocolos/TP3/capturas/lab4/vcd_pcap_originales/lab4_cafe0004.pcapng
[sudo] password for francisco-javier-vasquez:
tcpdump: listening on tap0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C12 packets captured
12 packets received by filter
0 packets dropped by kernel
+ -

```

```

+ ping -c 5 -p fec80004 -i tap0 10.0.0.2
PATTERN: 0xfeca0004
PING 10.0.0.2 (10.0.0.2) from 10.0.0.1 tap0: 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=215 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.078 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.083 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=31.4 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.071 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 407ms
rtt min/avg/max/mdev = 0.071/49.482/215.396/83.877 ms
+ -

```

Figura 9: Lab 4: emulador, `ping -c 5 -p fec80004` (5/5, 0 % de pérdida) y `tcpdump` (12 paquetes).

6.4 Respuestas a la consigna

Tarea 1 (preguntas 1.1 y 1.2). `mld_lane_id` incrementa de 0 a 19 y vuelve a 0 inmediatamente (figura 10). En cada flanco se despacha un bloque: `pcs_block_out` se actualiza en todos los ciclos, junto con `mld_lane_id`, sin huecos ni esperas. En la figura, los bloques de una trama ICMP (278D55... de *start*, 1CAFE0400CAFE0400 de datos, 2AA000... de *terminate*) pasan mientras el contador de lanes sigue corriendo.

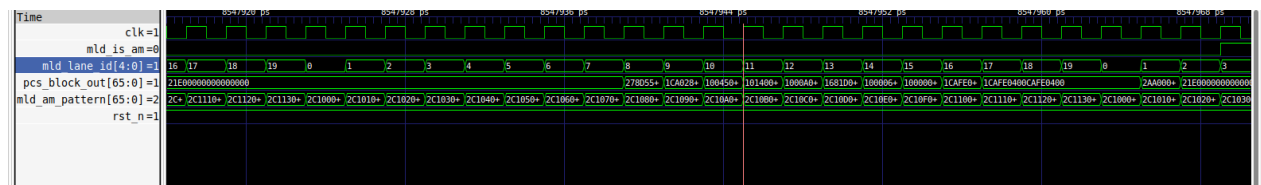


Figura 10: Lab 4, tarea 1: `mld_lane_id` (en decimal) recorre 0 a 19 mientras `pcs_block_out` transporta una trama ICMP.

Tarea 2 (preguntas 2.1 a 2.3).

- **2.1.** `mld_is_am` pulsa cada 64 ciclos: 128 ps entre pulsos, por ejemplo a 129 ps y a 257 ps (figura 11). Coincide con `-GAM_INTERVAL=64`. Sobre la traza completa, los pulsos siguen a 385, 513, 641, 769 ps, y así sucesivamente.
- **2.2.** Los lanes 0 y 5 **nunca** coinciden con un pulso de AM. El pulso k ocurre en el ciclo $64k$, cuando el lane es $(64k - 1) \bmod 20 = (4k - 1) \bmod 20$. Como $\gcd(64, 20) = 4$, solo aparecen los lanes **3, 7, 11, 15 y 19** (se repiten cada 5 pulsos, es decir, cada 320 ciclos), lo que se confirmó sobre la traza. Los valores capturados son `VL3 = 2C10300A5A5A5A5A5` y `VL7 = 2C10700A5A5A5A5A5`. Por fórmula, `VL0 = 2C10000A5A5A5A5A5` y `VL5 = 2C10500A5A5A5A5A5`.
- **2.3.** El campo que cambia con el lane es el byte `[55:48]` de `mld_am_pattern`, el que sigue a `C1`: contiene el número de lane (03, 07, ...). El resto del patrón es fijo: `{10, C1, lane, 00, A5 A5 A5 A5 A5}`.

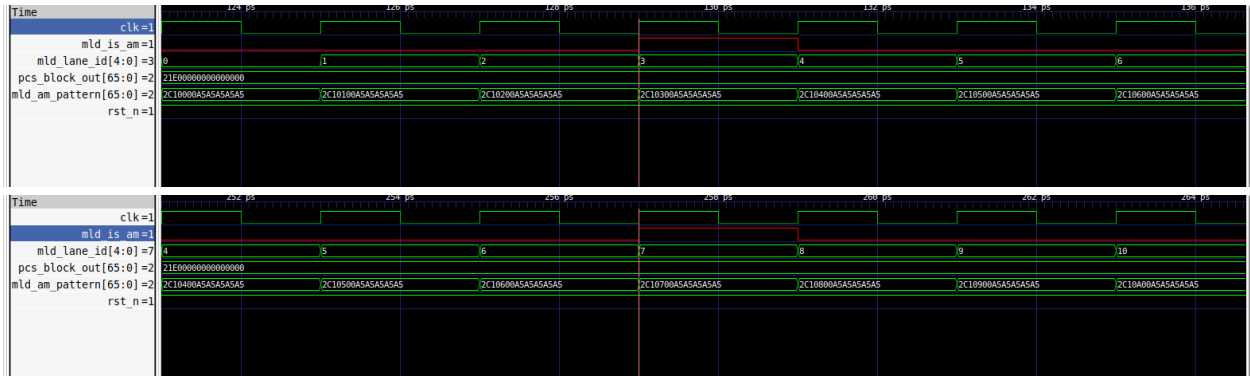


Figura 11: Lab 4, tarea 2: pulsos de `mld_is_am` a 129 ps (lane 3, arriba) y a 257 ps (lane 7, abajo), separados por 64 ciclos; el byte de lane en `mld_am_pattern` acompaña a `mld_lane_id`.

Tarea 3 (preguntas 3.1 y 3.2).

- **3.1.** El retardo es de **0 ciclos**. En `top.sv`, `mld_lane_id_a2b`, `mld_is_am_a2b` y `mld_am_pattern_a2b` conectan directamente las salidas de `mld_tx_inst_a` con las entradas de `mld_rx_inst_b`, sin registros intermedios. En la traza, ambos nombres jerárquicos comparten el mismo identificador: son la misma red.
- **3.2.** Sí. `pcs_block_out` y `mld_lane_id` se actualizan en el mismo `always_ff` de `mld_tx`, así que cada bloque llega a `mld_rx` junto con su etiqueta de lane. `mld_rx` no usa la banda lateral: solo desaleatoriza (si está habilitado) y registra el bloque, lo que agrega un ciclo al camino de datos.

Conclusión. El laboratorio muestra el mecanismo temporal de MLD (etiquetado *round-robin* y marcadores periódicos) sin la complejidad del *deskew*. La elección de parámetros tiene una consecuencia no documentada: con `AM_INTERVAL` múltiplo de 4 y 20 lanes, solo 5 lanes llegan a “ver” un AM. En el estándar esto no ocurre, porque los AM se insertan en todos los lanes simultáneamente.

7 Conclusiones generales

- **Cada subcapa cambia la representación en el enlace para resolver un problema físico.** Los datos pasan de bytes (lab 1) a palabras de 64 bits con control fuera de banda (lab 2), luego a bloques de 66 bits aleatorizados (lab 3) y por último a 20 lanes virtuales con marcadores de alineación (lab 4). Los problemas que se resuelven son, en ese orden: ancho de bus frente a frecuencia, delimitación de tramas, densidad de transiciones y balance de continua, y reparto sobre carriles paralelos con *skew*.
- **Las subcapas son transparentes.** En los cuatro labs `ping` funcionó sin pérdidas y Wireshark mostró las mismas tramas: mismas MAC, EtherType y payload. Las diferencias solo aparecen en la traza del RTL. Por eso el método central del TP fue correlacionar el volcado hexadecimal de Wireshark con los valores ciclo a ciclo en GTKWave, teniendo en cuenta el orden de lanes.

- **Leer el RTL fue indispensable.** Los nombres de señales documentados no coincidían con los reales, el lab 2 cambió de versión durante el curso y el lab 3 no volcaba la traza. En todos los casos, interpretar las formas de onda requirió partir del código real.
- **Ancho de bus y ciclos.** La misma trama de 98 bytes ocupa 98 ciclos con el bus de 8 bits y 14 con el de 64 bits. Esto se refleja incluso en el RTT de **ping**: 10–15 ms en el lab 1 frente a 1,5–2,3 ms en el lab 2, una relación cercana a $98/14 = 7$. Ambos wrappers hacen una pausa (**usleep**) en cada ciclo simulado, así que el RTT mide la velocidad de la simulación y no la de un enlace físico. En los labs 3 y 4, sin esa pausa, el RTT baja típicamente a menos de 0,25 ms.